Polymorphism

Tuesday, October 14, 2025 4:22 PM

Polymorphism in Python

Definition:

- Polymorphism means **"one object showing different behaviors"**.
- poly → many, morphism → forms
- **Purpose:** To make a method or operator work in **different ways depending on context or data types**.

Types of Polymorphism:

A. Method Overloading

- **Definition:** Creating multiple versions of the same function/method **with different arguments**.
- **Purpose:** Make a single method name work in multiple ways.

Example in Python:

```
def sample(a, b):
    print(a + b)
def sample(a, b, c):
    print(a * b * c)
# Note: Python does **not support traditional method overloading**.
# If multiple functions have the same name, the last one **overrides** the previous ones.
```

B. Method Overriding

- **Definition:** When a child class redefines a method from the parent class with the **same name**, it **overrides** the parent method.

Example:

```
class Parent:
    def show(self):
        print("Parent method")
class Child(Parent):
    def show(self):
        print("Child method")
c = Child()
c.show()  # Output: "Child method"
```

C. Monkey Patching

- **Definition:** Dynamically modifying or replacing a method at runtime.
- Can assign a method to a variable and call it like a function.
- **Optional:** Not mandatory to store in variable or call it.

Example:

```
def sample():
    print("Original method")
l = sample()  # storing return
```

Operator Overloading

- **Definition:** Using the same operator to perform **different operations** depending on data types.

Example:

```
10 + 10       # sum
'Hello' + 'Hi' # concatenation
'10' + '20'    # string concatenation
```

- For **user-defined classes**, we can make operators like +, -, * work by defining **magic methods**.

Syntax for Operator Overloading

```
class ClassName:
    def __init__(self, value):
        self.value = value
def __add__(self, other):  # '+' operator overloading
        return self.value + other.value
obj1 = ClassName(10)
obj2 = ClassName(20)
print(obj1 + obj2)  # Calls obj1.__add__(obj2)
```

Example: Overloading + for multiple attributes

```
class Demo:
    def __init__(self, a, b, c):
        self.a = a
        self.b = b
```

```
        self.c = c
    def __add__(self, other):
        return self.a + other.a, self.b + other.b
obj1 = Demo(100, 3, 4)
obj2 = Demo(200, 5, 4)
print(obj1 + obj2)  # Output: (300, 8)
```

Bank Account Example

```
class Bank:
   def __init__(self,details):
      self.details=details
   def __add__(self, other):
       return Bank()
Ac1 = Bank([100000, 8392989878826, 'Alice', '864387678'])
Ac2 = Bank(50000, 1234567890123, 'Bob', '987654321')
print(Ac1 + Ac2)
```

Output:

```
(150000, 9627548768949, 'AliceBob', '864387678987654321')
```

Cart Example: Operator Overloading

```
class Cart:
   def __init__(self, items=()):   #  Correct: use __init__ (not _init_)
      self.items = items

   def __add__(self, other):       #  Optional: define addition behavior
      return Cart(self.items + other.items)

# Creating objects
cloths = Cart(['formal', 'panjabi', 'babydresses', 'T-shirts', 'shirts', 'jacket',
'sweaters'])
toys = Cart(['teddybears', 'cartoons', 'peppa', 'carroms', 'plasticcar', 'piggybanks',
'chibifigure'])
vehicle = Cart(['helicopter', 'bikes', 'cars', 'Jcb', 'Scooty', 'Electric scooter', 'bus',
'tractor', 'bicycle'])

# Using + operator (thanks to __add__)
```

```
flipcart = cloths + toys
new_cart = flipcart + vehicle

# Display results
print(new_cart.items)
```

Explanation Step-by-Step

1. __init__ Method

- The constructor must be written as **__init__** (double underscore at both sides).
- You wrote _init_, which Python doesn't recognize.
- This method initializes the object with items.

```
def __init__(self, items=()):
    self.items = items
```

2. __add__ Method (Operator Overloading)

- This method defines what happens when you use the + operator between two objects.
- Here, you're combining the items from two carts into one new cart.

Example:

```
def __add__(self, other):
    return Cart(self.items + other.items)
```

So when you do:

```
flipcart = cloths + toys
```

→ Internally Python calls

Cart.__add__(cloths, toys)

and returns a new Cart object with both item lists combined.

3. Object Creation and Combination

Each object holds a list of items:

```
cloths.items  → ['formal', 'panjabi', ...]
toys.items    → ['teddybears', 'cartoons', ...]
vehicle.items → ['helicopter', 'bikes', ...]
```

When you do:

flipcart = cloths + toys
new_cart = flipcart + vehicle
→ It combines all 3 lists.

4. Output
Final output:

['formal', 'panjabi', 'babydresses', 'T-shirts', 'shirts', 'jacket', 'sweaters', 'teddybears', 'cartoons', 'peppa', 'carroms', 'plasticcar', 'piggybanks', 'chibifigure', 'helicopter', 'bikes', 'cars', 'Jcb', 'Scooty', 'Electric scooter', 'bus', 'tractor', 'bicycle']

Key Notes

1. **Polymorphism = Same name, different behavior**
2. **Method Overloading**: Same method name, different arguments (Python uses overriding).
3. **Method Overriding**: Child class redefines parent method.
4. **Operator Overloading**: Make operators (+, -, *) work for **user-defined classes**.
5. **Magic Methods** (__add__, __sub__, __mul__, etc.) are used for operator overloading.